Software Requirements Specification (SRS)

Digital Library and Booking System (DLBS)

Project: Digital Library and Booking System (DLBS)

Version: 1.0

Authors: Mohammed Sinan M T, Mallela Rishyendra, Nagalapuram Sai Mukesh Reddy, Marreddy Rushieswar Reddy

Date: 30-10-2026

Status: Submitted (baseline before Sprint 1)

Revision history

Version	Date	Author	Change summary	Approval
1.0	30-10-2026	Team DLBS (all four members)	Baseline SRS for Sprint 1: requirements, interfaces, use-case diagrams, RTM	Pending

Approvals

Role	Name	Signature / Email	Date
Course Coordinator			
Scrum Master	Mallela Rishyendra		
QA Lead	Marreddy Rushieswar Reddy		

Table of Contents

1. Introduction 3
1.1 Purpose 3
1.2 Scope 3
1.3 Audience 3
1.4 Definitions 3
1.5 Project team and schedule 3
2. Overall description 4
2.1 Product perspective 4
2.2 Major product functions 4
2.3 User roles and characteristics 4
2.4 Operating environment 4
2.5 Constraints 4
2.6 Assumptions and dependencies 4
3. External interface requirements 5
3.1 User interfaces 5
3.2 Hardware interfaces 5
3.3 Software interfaces 5
3.4 Communications 5
4. System features (detailed) 6
4.1 Book Catalog 6
4.2 User Accounts and Reading Tracking 6
4.3 Booking and Borrowing 7
4.4 Personalised Recommendations 8
5. Non-functional requirements (detailed) 8
5.1 Security 9
6. Quality attributes and acceptance tests 10
7. System models and diagrams 11
7.1 UML use-case diagrams 11
8. Requirements Traceability Matrix (RTM) 14

1. Introduction

1.1 Purpose

This document is the Software Requirements Specification for the Digital Library and Booking System. It defines the functional, non-functional and security requirements, the external interfaces and the acceptance criteria of the system. It is the baseline for the two development sprints, for the Software Architecture and Design Specification (SAD) and for the Software Test Plan (STP).

1.2 Scope

The Digital Library and Booking System is a standalone application for managing a digital book collection. It covers book cataloging, booking and borrowing of copies, user reading history and progress tracking, and personalised book recommendations produced by machine-learning algorithms.

In scope: book catalog management and search; user accounts with member and administrator roles; reading status, progress, history and statistics; reserving, borrowing, returning and cancelling copies with due dates; personalised recommendations with a reason for each; the CI/CD pipeline and deployment of the application.

Out of scope: payments and fines; e-mail or SMS notifications; hosting or displaying e-book files inside the application; password reset by e-mail; mobile apps; integration with external library systems.

1.3 Audience

The development team, the QA lead, the Scrum Master, maintainers of the deployed application, and the course instructor and teaching assistants who assess the project.

1.4 Definitions

Term	Meaning
DLBS	Digital Library and Booking System
FR / NFR / SR	Functional, non-functional and security requirement
SO	Security objective
API / REST	Application programming interface; HTTP interface using JSON resources
JWT	JSON Web Token: the signed login token sent as Authorization: Bearer <token>
TF-IDF	Term frequency – inverse document frequency, used to compare book descriptions
Cold start	Recommending to a user who has no reading history yet
Soft delete	Marking a record as removed instead of deleting it, so history stays intact
Hold / reservation	A copy kept for a member for up to 2 days before it is borrowed
CI/CD	Continuous integration and continuous deployment (GitHub Actions)
RTM	Requirements Traceability Matrix
ISBN	International Standard Book Number

1.5 Project team and schedule

Member	Name	Module owned	Team role
Member 1	Mohammed Sinan M T	Recommendation Engine	Architecture owner, presentation lead
Member 2	Mallela Rishyendra	Book Catalog, app shell and home page	Scrum Master
Member 3	Nagalapuram Sai Mukesh Reddy	Users and Reading Tracking	DevOps (repository, CI/CD, deployment)
Member 4	Marreddy Rushieswar Reddy	Booking and Borrowing	QA Lead

Sprint 1 runs 30 Oct – 3 Nov 2026 and Sprint 2 runs 3 Nov – 8 Nov 2026; presentations take place from 9 Nov 2026. Work is tracked on the team's Jira board and in one GitHub repository.

2. Overall description

2.1 Product perspective

DLBS is a new, self-contained web application. A browser-based user interface calls a REST API. Behind the API, four modules (Catalog, Users and Reading, Booking, Recommendation) share one relational database and call each other in-process. The whole system is one deployable unit with its own database and no outside services. GitHub Actions builds, tests and deploys it to a free cloud host.

2.2 Major product functions

- Book catalog: add, edit and remove books (administrators); search, filter, paginate and view details.
- Accounts: register, log in and out, member and administrator roles, view and edit the profile.
- Reading tracking: set a reading status, record progress, view history and statistics.
- Booking and borrowing: reserve, borrow (14-day loan), return, cancel; overdue flags; 2-day hold expiry; at most 5 active bookings; no double booking.
- Recommendations: ranked personalised suggestions from reading history using TF-IDF content similarity and popularity, a cold-start fallback, and a reason for every suggestion.
- Operations: automated build, lint, test and deployment with a /health smoke test.

2.3 User roles and characteristics

Role	Characteristics	Main goals
Visitor	Not logged in; any browser user	Browse and search the catalog; register
Member	Registered reader; basic computer skills; uses the app weekly	Track reading, reserve and borrow books, get good suggestions
Administrator	Library staff; trusted; few users	Keep the catalog accurate; refresh recommendations

2.4 Operating environment

- Server: Python 3.11 application in a Linux Docker container on a free cloud host (for example Render or Railway); PostgreSQL in Docker and on the host; SQLite for local development.
- Client: current versions of Chrome, Edge and Firefox on desktop and mobile screen sizes.
- Development: Windows, macOS or Linux with Python 3.11; GitHub for code; Jira for planning.

2.5 Constraints

- Two sprints only (30 Oct – 3 Nov 2026 and 3 Nov – 8 Nov 2026) and a team of four.
- One GitHub repository for all code and the three project deliverables; main is protected and changes arrive through reviewed pull requests.
- Python stack: FastAPI, SQLAlchemy, scikit-learn; free hosting tier; no paid external services.
- Book data from the public Goodbooks-10k dataset, with generated values for missing fields; demo users and reading records are synthetic.

2.6 Assumptions and dependencies

- Goodbooks-10k remains downloadable and its licence allows academic use.

- A free host with Docker support is available for the demo; a local Docker Compose run is the fallback.
- Each module owner publishes the interfaces in docs/api-contract.md before other modules depend on them.

3. External interface requirements

3.1 User interfaces

Screen	Purpose	Requirements
Register / Log in	Create an account; sign in and out	FR-U1, FR-U2
Home	Recommendation cards with the reason under each	FR-C9, FR-R1–R5
Catalog	Search box, genre and availability filters, paging	FR-C4, FR-C5, FR-C7
Book detail	Book data; Reserve or Borrow; reading status and progress	FR-C6, FR-B1, FR-U5, FR-U6
My Reading	History with status and a progress bar; statistics	FR-U7, FR-U8
My Bookings	Status badges, due dates, overdue flag; Return and Cancel	FR-B3, FR-B4, FR-B6, FR-B7
Profile	View and edit name	FR-U4
Admin: books	Add, edit and remove books; refresh recommendations	FR-C1–C3, FR-R6

Accessibility: every form field has a visible label, all actions work with the keyboard, images have alternative text, status is never shown by colour alone, and contrast meets WCAG 2.1 AA where applicable (NFR-G2).

3.2 Hardware interfaces

No special hardware. The server runs on standard cloud or desktop hardware; users need any device with a modern browser.

3.3 Software interfaces

Interface	Type	Provided by	Used by
REST API /api/books, /api/auth, /api/users, /api/reading, /api/bookings, /api/recommendations, /health	HTTP + JSON	Each module's router	Web UI, tests, smoke test
catalog.get_books(), get_book(id), get_availability(id)	Python, in-process	Catalog	Booking, Recommendation
get_current_user(), require_admin, create_user(), reading.get_history(user_id)	Python, in-process	Users and Reading	All modules, seed script
Recommender.recommend(user_id, n)	Python interface	Recommendation	Recommendation API
Relational database	SQLAlchemy ORM	SQLite / PostgreSQL	All modules
GitHub Actions; hosting platform	CI/CD service	GitHub; host	Repository

3.4 Communications

- HTTPS on the deployed host (TLS terminated by the platform); JSON bodies in UTF-8.
- Authorization: Bearer <JWT> on every protected request; tokens expire after 60 minutes.
- Errors use the body {"detail": "message"} with standard status codes (401, 403, 404, 409, 422).

This SRS contains 33 functional requirements, 15 non-functional requirements, 3 security objectives and 7 security requirements (the template asks for at least 15, 5, 2 and 5).

4. System features (detailed)

Each requirement below has acceptance criteria and the test cases that verify it (test cases are defined in the STP). IDs follow FR-<module><number>: C catalog, U users and reading, B booking, R recommendations.

4.1 Book Catalog

Description: Administrators maintain the books; everyone can find and inspect them.

Req ID	Requirement (shall …)	Type	Priority	Source / Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
FR-C1	An administrator shall be able to add a book with title, author, ISBN, genre, description, publication year, number of pages, cover image URL and number of copies.	Functional	High	Administrator	A valid book is saved with available copies equal to total copies (201) and then appears in search. Test: TC-C01	Admin role (FR-U3, FR-C8)
FR-C2	An administrator shall be able to edit a book's details. Changing the number of copies shall change the available copies by the same amount; a change that would make them negative shall be rejected.	Functional	High	Administrator	Edits are saved; raising copies from 3 to 5 adds 2 available copies; cutting below the copies on loan returns 409. Test: TC-C02, TC-C03	Booking changes available copies (FR-B5)
FR-C3	An administrator shall be able to remove a book (soft delete). A removed book disappears from search and recommendations and cannot be reserved, but its existing bookings can still be returned.	Functional	High	Administrator	Book flagged deleted; absent from search and recommendations; reserve rejected; open loan can be returned. Test: TC-C04	FR-B1, FR-R4
FR-C4	Any user shall be able to search books by title, author or ISBN.	Functional	High	Member, Visitor	Partial, case-insensitive match on title or author and exact match on ISBN; no match returns an empty list. Test: TC-C05	NFR-C1
FR-C5	Any user shall be able to filter books by genre and availability.	Functional	Medium	Member, Visitor	genre=Fantasy&available=true returns only Fantasy books with at least one available copy. Test: TC-C06	FR-B5
FR-C6	Any user shall be able to open a book detail page.	Functional	High	Member, Visitor	All book fields and the current available copies are shown; an unknown id returns 404. Test: TC-C07	–
FR-C7	Book lists shall be paginated.	Functional	Medium	Member, Visitor	page and size parameters; response holds items, page and total; size is capped at 50. Test: TC-C08	–
FR-C8	Only users with the admin role may add, edit or delete books.	Functional	High	Library policy	A member gets 403 and a request without a token gets 401 on POST, PUT and DELETE /api/books. Test: TC-C09	FR-U3, SR-03
FR-C9	The home page shall show the logged-in user's top recommendations as book cards, with the reason text under each card.	Functional	Medium	Member	Up to 10 cards from GET /api/recommendations/me, each with cover, title, author and reason. Test: TC-C10	FR-R1, FR-R5

4.2 User Accounts and Reading Tracking

Description: People create accounts and log in; members track what they read.

Req ID	Requirement (shall …)	Type	Priority	Source / Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
FR-U1	A visitor shall be able to register with name, email and password.	Functional	High	Visitor	Creates a member account (201) without returning the password; duplicate email returns 409; invalid email or weak password returns 422. Test: TC-U01, TC-U02, TC-U03	NFR-U1, NFR-U3

Req ID	Requirement (shall …)	Type	Priority	Source / Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
FR-U2	A registered user shall be able to log in and log out.	Functional	High	Member	Correct credentials return a signed token valid for 60 minutes; wrong email or password returns the same 401 message; logging out removes the token. Test: TC-U04, TC-U05, TC-U06	SR-02
FR-U3	The system shall support two roles: member and admin.	Functional	High	Library policy	New accounts are members; the first admin is created from ADMIN_EMAIL and ADMIN_PASSWORD; admin-only actions return 403 to members. Test: TC-U07	FR-C8, FR-R6
FR-U4	A user shall be able to view and edit their profile.	Functional	Medium	Member	GET /api/users/me returns id, name, email and role; PUT changes the name only. Test: TC-U08	–
FR-U5	A user shall be able to set a reading status for a book: want to read, reading, finished.	Functional	High	Member	Status saved per user and book; unknown status returns 422; unknown book returns 404. Test: TC-U09	Book exists in Catalog
FR-U6	A user shall be able to record reading progress (page number or percentage) for a book.	Functional	High	Member	Page 120 of 320 stores 37.5%; below 0, above 100% or past the last page returns 422; 100% marks the book finished. Test: TC-U10, TC-U11	page_count from Catalog
FR-U7	A user shall be able to view their reading history.	Functional	High	Member, Recommendation Engine	Records newest first with title, author, genre, status and progress; a new user gets an empty list; reading.get_history(user_id) returns the same data in-process. Test: TC-U12	FR-R2
FR-U8	A user shall be able to see simple reading statistics (books finished, pages read).	Functional	Medium	Member	Counts by status and pages read (sum of current pages) match the user's records. Test: TC-U13	–

4.3 Booking and Borrowing

Description: Members reserve, borrow and return copies under clear rules.

Req ID	Requirement (shall …)	Type	Priority	Source / Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
FR-B1	A logged-in member shall be able to reserve a book that has at least one available copy and has not been removed from the catalog.	Functional	High	Member	Booking created as reserved and available copies drop by one; a removed book is rejected. Test: TC-B01, TC-B06	FR-C3, catalog.get_availability()
FR-B2	A member shall be able to borrow a reserved book, which sets a due date 14 days ahead.	Functional	High	Member	Status becomes borrowed, due date is 14 days after borrowing, available copies unchanged. Test: TC-B07	FR-B1
FR-B3	A member shall be able to return a borrowed book, which makes the copy available again.	Functional	High	Member	Status becomes returned and available copies rise by one; returning a booking that is not borrowed is rejected. Test: TC-B03, TC-B12	FR-B2
FR-B4	A member shall be able to cancel a reservation they have not yet borrowed.	Functional	Medium	Member	Status becomes cancelled and available copies rise by one; a borrowed booking cannot be cancelled. Test: TC-B08	FR-B1
FR-B5	The available-copy count shall update automatically on reserve, cancel, return and expiry; borrowing leaves it unchanged.	Functional	High	Library policy	Counts checked after every action in TC-B01, TC-B03, TC-B05 and TC-B08. Test: TC-B01, TC-B03, TC-B05, TC-B08	Only booking code changes available_copies

Req ID	Requirement (shall …)	Type	Priority	Source / Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
FR-B6	The system shall flag a booking as overdue when the due date has passed.	Functional	Medium	Member, Administrator	A borrowed booking read after its due date shows overdue = true. Test: TC-B09	FR-B2
FR-B7	A member shall be able to view their active and past bookings.	Functional	Medium	Member	GET /api/bookings/me lists only the member's bookings with status, dates and overdue flag. Test: TC-B10	–
FR-B8	A member shall not hold more than 5 active bookings at once.	Functional	Medium	Library policy	A sixth active reservation or loan is rejected with 409. Test: TC-B04	–
FR-B9	A member shall not be able to book a copy that is unavailable (no double booking).	Functional	High	Library policy	Reserving a book with 0 available copies returns 409 and changes nothing. Test: TC-B02	NFR-B1
FR-B10	A reservation that is not borrowed within 2 days shall expire, and its copy shall become available again.	Functional	Medium	Library policy	A reservation read more than 2 days after it was made shows expired and its copy is available again. Test: TC-B05	FR-B5

4.4 Personalised Recommendations

Description: Members get ranked, explained suggestions based on their reading.

Req ID	Requirement (shall …)	Type	Priority	Source / Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
FR-R1	The system shall generate a ranked list of top-N book recommendations for a logged-in user.	Functional	High	Member	GET /api/recommendations/me?limit=10 returns at most 10 items sorted by score with book_id, title, author, cover_url, score and reason; no token returns 401. Test: TC-R01, TC-R08	FR-C9
FR-R2	The system shall use the user's reading history (books read, progress, genres) to personalise results.	Functional	High	Member	For a user who finished several books of one genre, at least 3 of the top 5 share it. Test: TC-R02	reading.get_history(user_id)
FR-R3	For a new user with no history, the system shall fall back to popular or genre-based recommendations (cold start).	Functional	High	Member	A user with no records gets the most popular books, not an error or an empty list. Test: TC-R03	–
FR-R4	The system shall not recommend books the user has already finished or books removed from the catalog.	Functional	High	Member	No finished or removed book appears in any result. Test: TC-R04	FR-C3
FR-R5	The system shall show a short reason for each recommendation (for example: "Because you read X").	Functional	Medium	Member	Every item has a non-empty reason. Test: TC-R05	FR-C9
FR-R6	An administrator shall be able to trigger a refresh or retrain of the model.	Functional	Medium	Administrator	POST /api/recommendations/refresh by an admin rebuilds the model and clears the cache; a member gets 403. Test: TC-R06	FR-U3

5. Non-functional requirements (detailed)

Each NFR is measurable and tied to a test case. IDs follow NFR-<module><number>; G marks system-wide requirements.

Req ID	Requirement	Category	Priority	Acceptance criteria / Measurement
NFR-C1	Search shall return results in under 1 second for 10,000 books.	Performance	High	95th percentile below 1 s over 20 searches on the 10,000-book seed. Test: TC-P01

Req ID	Requirement	Category	Priority	Acceptance criteria / Measurement
NFR-C2	ISBN and required fields shall be validated, and duplicate ISBNs rejected.	Data integrity	High	Missing field or invalid ISBN returns 422; duplicate ISBN returns 409. Test: TC-C11, TC-C12
NFR-C3	The catalog module shall have at least 70% unit-test coverage.	Maintainability	Medium	pytest-cov report for src/catalog at or above 70%. Test: TC-Q01
NFR-U1	Passwords shall be stored hashed (bcrypt or argon2), never in plain text.	Security	High	Stored value is a bcrypt hash; the plain password is in no table or log. Test: TC-S01
NFR-U2	All protected endpoints shall reject requests without a valid token or session.	Security	High	Missing, tampered or expired token returns 401 on every protected endpoint. Test: TC-S02, TC-S03
NFR-U3	Email and password input shall be validated on the server.	Security	High	Invalid email or weak password returns 422 even when the UI checks are bypassed. Test: TC-U03
NFR-U4	The Users and Reading module shall have at least 70% unit-test coverage.	Maintainability	Medium	pytest-cov report for src/users at or above 70%. Test: TC-Q01
NFR-B1	Booking actions shall be atomic: two users booking the last copy at the same time must not both succeed.	Reliability	High	In a concurrent test exactly one reservation succeeds and available copies never go below 0. Test: TC-B11
NFR-B2	Every booking action shall complete in under 1 second.	Performance	Medium	Each of 20 timed reserve, borrow and return calls takes under 1 s. Test: TC-P02
NFR-B3	The booking module shall have at least 70% unit-test coverage.	Maintainability	Medium	pytest-cov report for src/booking at or above 70%. Test: TC-Q01
NFR-R1	A recommendation request shall respond in under 2 seconds for 10,000 books and 1,000 users.	Performance	High	Each of 20 uncached requests on the full seed takes under 2 s. Test: TC-P03
NFR-R2	The algorithm shall sit behind a common interface so it can be swapped without changing other modules.	Maintainability	High	A different Recommender implementation is plugged in with no change outside src/recommender. Test: TC-R07
NFR-R3	The recommendation module shall have at least 70% unit-test coverage.	Maintainability	Medium	pytest-cov report for src/recommender at or above 70%. Test: TC-Q01
NFR-G1	Every push and pull request shall run build, lint and tests in GitHub Actions, and every merge to main shall deploy automatically and pass a /health smoke test.	Deployability	High	CI result shown on every pull request; a failed stage blocks the merge; deployed /health returns 200. Test: TC-Q02
NFR-G2	The web UI shall be usable with a keyboard and a screen reader, meeting WCAG 2.1 AA contrast where applicable.	Usability / Accessibility	Medium	Every form field has a label, every action is reachable by keyboard, and the Lighthouse accessibility score is at least 90. Test: TC-UX01

5.1 Security

5.1.1 Security objectives

- **SO-1 Confidentiality:** Only the account owner, and administrators where the role requires it, can see a user's account, reading history and bookings. Passwords and secrets are never exposed.
- **SO-2 Integrity:** Catalog data changes only through administrators, and copy counts and bookings change only through the booking rules, so stock never goes negative and no copy is double-booked.
- **SO-3 Authenticated access:** Every protected action is tied to an authenticated user and their role through short-lived signed tokens.

5.1.2 Security requirements

Req ID	Requirement (shall …)	Type	Priority	Acceptance criteria / Test case ref
SR-01	The system shall store passwords only as salted bcrypt hashes with a cost factor of 12 or more.	Security	High	Stored value starts with $2b$12$; the plain password appears nowhere. Test: TC-S01
SR-02	Every protected endpoint shall require a signed JWT that has not expired (60-minute lifetime); a missing, tampered or expired token shall be rejected with 401.	Security	High	401 for all three cases on every protected endpoint. Test: TC-S02, TC-S03
SR-03	Administrator-only endpoints (book changes, recommendation refresh) shall return 403 to members.	Security	High	Member token gets 403 on every admin endpoint. Test: TC-S05, TC-C09
SR-04	A member shall only be able to read or change their own reading records and bookings.	Security	High	Requests for another member's records return 403 or 404 and change nothing. Test: TC-S06
SR-05	All input shall be validated on the server, and database access shall use the ORM's parameterised queries so that input can never change a query.	Security	High	Injection strings are treated as plain text; invalid input returns 422. Test: TC-S04, TC-C12
SR-06	Secrets (JWT secret, admin password, database URL, deploy tokens) shall be kept only in environment variables or GitHub Secrets and never committed to the repository.	Security	High	Repository scan finds no secrets; .env is git-ignored; CI reads GitHub Secrets. Test: TC-S07
SR-07	The deployed application shall be served over HTTPS only.	Security	Medium	http:// redirects to https:// with a valid certificate. Test: TC-S08

6. Quality attributes and acceptance tests

Quality attribute	How DLBS achieves it	Requirements
Performance	Indexed search, paging, cached recommendations, single-transaction bookings	NFR-C1, NFR-B2, NFR-R1
Security	bcrypt hashes, signed short-lived tokens, role and ownership checks, ORM queries, secrets outside the code	NFR-U1–U3, SR-01–SR-07
Reliability	Atomic reservation of the last copy; state rules for bookings	NFR-B1, FR-B9
Maintainability	Modules behind small interfaces; 70% unit-test coverage per module	NFR-R2, NFR-C3, NFR-U4, NFR-B3, NFR-R3
Deployability	CI on every push and pull request; automatic deploy and smoke test	NFR-G1
Usability	Simple screens, clear messages, keyboard and screen-reader support	NFR-G2

Exit criteria for acceptance: all High-priority functional requirements implemented and verified; no open critical or high defects; every module at or above 70% unit-test coverage; the pipeline green on main; the application deployed and passing its smoke test; and the RTM showing every planned test case passed.

Acceptance test suites: Catalog, Accounts and Reading, Booking, Recommendations, Performance, Security, and Accessibility. The suites, levels and schedule are in the STP.

7. System models and diagrams

7.1 UML use-case diagrams

Two diagrams cover all functional requirements. Use-case names carry the IDs of the requirements they realise. A Member is a specialised Visitor; administrators use the catalog and recommendation administration use cases.

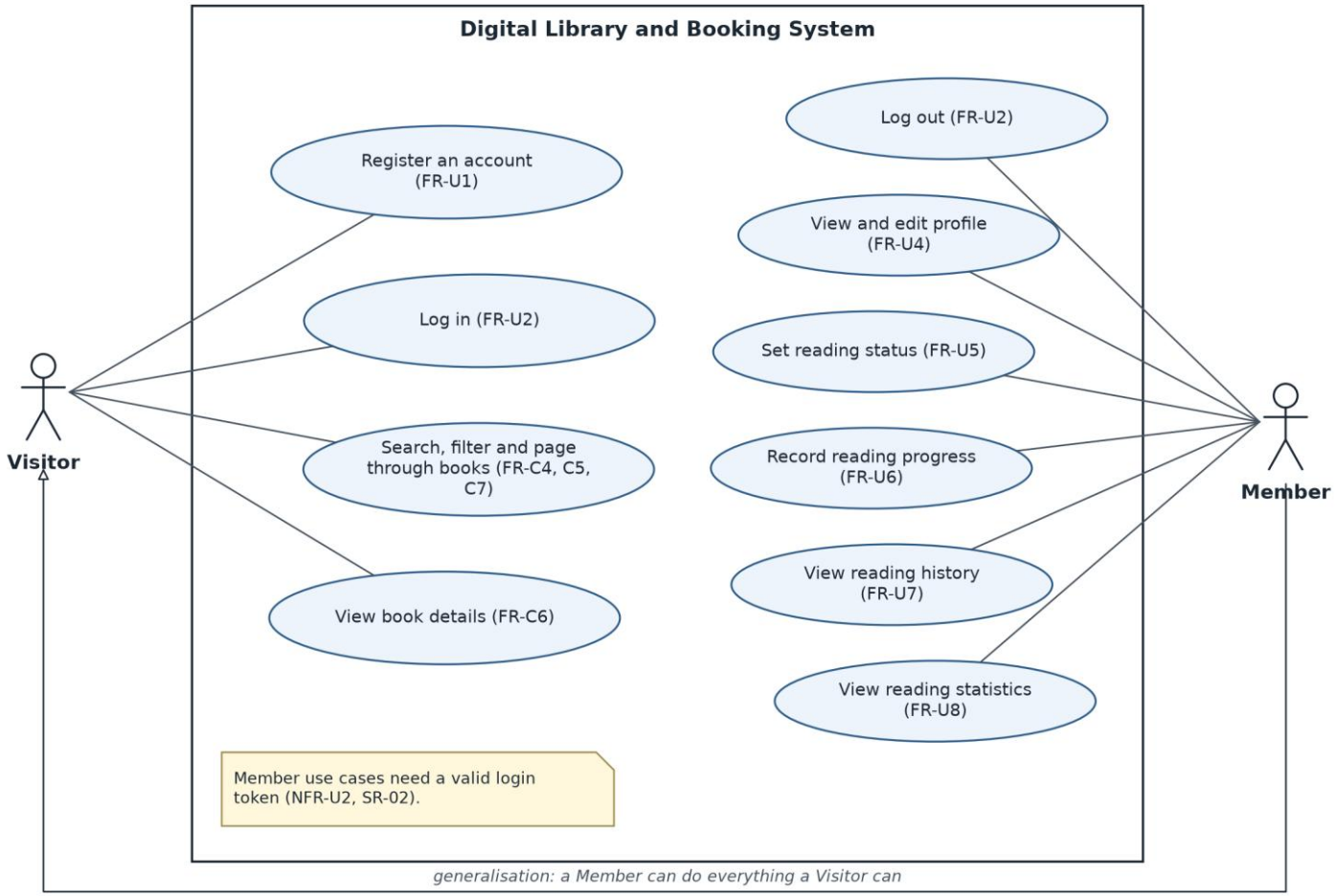


Figure 1. Use cases for accounts, catalog browsing and reading tracking (Visitor and Member).

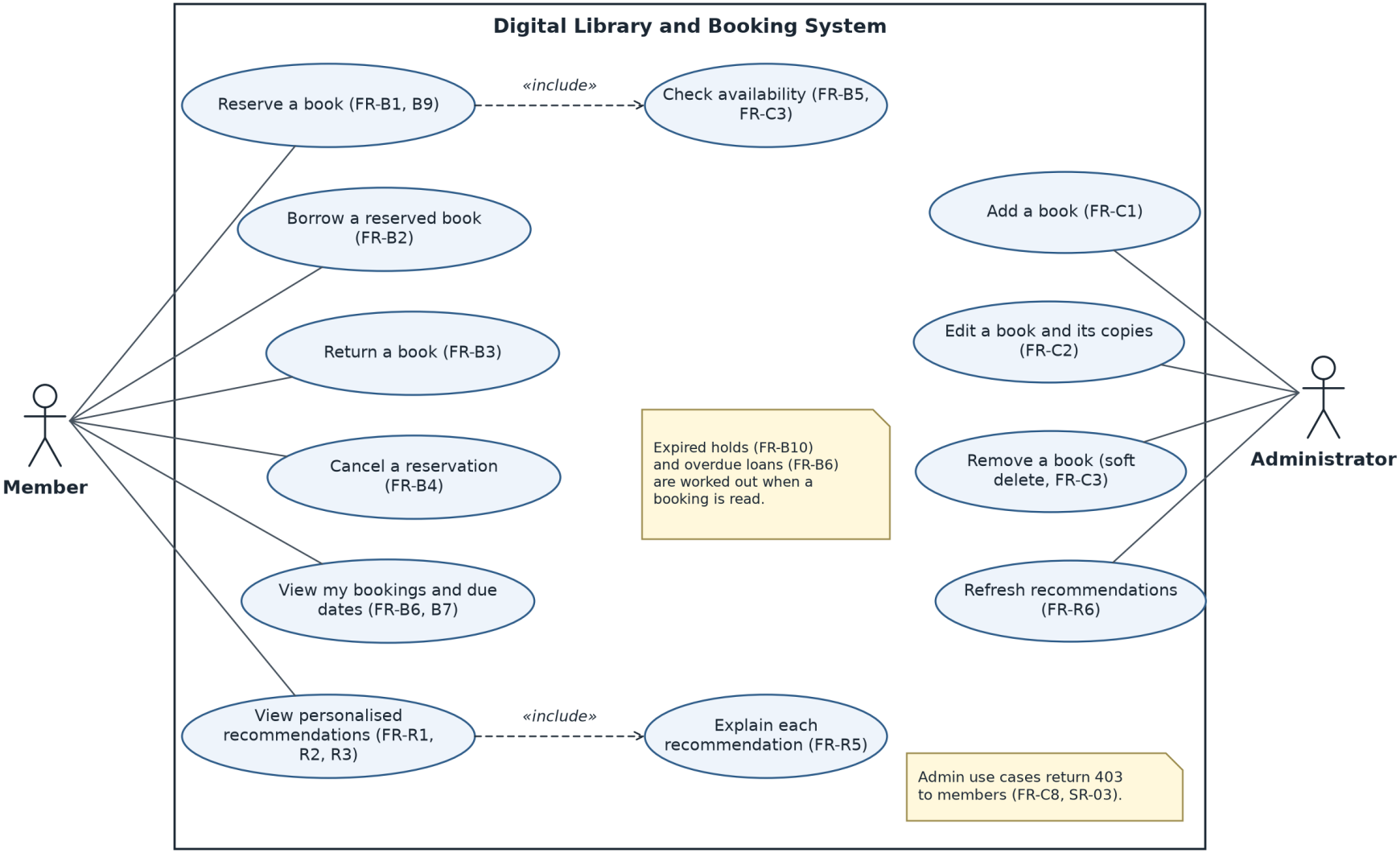


Figure 2. Use cases for booking, recommendations and administration (Member and Administrator).

8. Requirements Traceability Matrix (RTM)

Status: N = not yet tested, P = passed, A = accepted in a sprint review. Every requirement maps to at least one test case. The live copy of this matrix is kept by the QA Lead in docs/qa/rtm.md.

Req ID	Requirement (short)	Section ref / Design spec	Module	Test case(s)	Status	Owner / sprint
FR-C1	An administrator can add a book with title, author, ISBN, …	4.1 / SAD 3.4 Catalog; 4.3	Catalog	TC-C01	N	M2 Rishyendra; Sprint 1
FR-C2	An administrator can edit a book's details. Changing the …	4.1 / SAD 3.4 Catalog; 4.3	Catalog	TC-C02, TC-C03	N	M2 Rishyendra; Sprint 1
FR-C3	An administrator can remove a book (soft delete). A …	4.1 / SAD 3.4 Catalog; 4.3	Catalog	TC-C04	N	M2 Rishyendra; Sprint 1
FR-C4	Any user can search books by title, author or ISBN.	4.1 / SAD 3.4 Catalog; 4.3	Catalog	TC-C05	N	M2 Rishyendra; Sprint 1
FR-C5	Any user can filter books by genre and availability.	4.1 / SAD 3.4 Catalog; 4.3	Catalog	TC-C06	N	M2 Rishyendra; Sprint 2
FR-C6	Any user can open a book detail page.	4.1 / SAD 3.4 Catalog; 4.3	Catalog	TC-C07	N	M2 Rishyendra; Sprint 1–2
FR-C7	Book lists shall be paginated.	4.1 / SAD 3.4 Catalog; 4.3	Catalog	TC-C08	N	M2 Rishyendra; Sprint 1
FR-C8	Only users with the admin role may add, edit or delete …	4.1 / SAD 3.4 Catalog; 4.3	Catalog	TC-C09	N	M2 Rishyendra; Sprint 1
FR-C9	The home page shall show the logged-in user's top …	4.1 / SAD 3.4 Catalog; 4.3	Catalog	TC-C10	N	M2 Rishyendra; Sprint 2
FR-U1	Visitor can register with name, email and password.	4.2 / SAD 3.4 Users; 4.2 SD-1; 4.3	Users and Reading	TC-U01, TC-U02, TC-U03	N	M3 Mukesh; Sprint 1
FR-U2	User can log in and log out.	4.2 / SAD 3.4 Users; 4.2 SD-1; 4.3	Users and Reading	TC-U04, TC-U05, TC-U06	N	M3 Mukesh; Sprint 1
FR-U3	support two roles: member and admin.	4.2 / SAD 3.4 Users; 4.2 SD-1; 4.3	Users and Reading	TC-U07	N	M3 Mukesh; Sprint 1
FR-U4	A user can view and edit their profile.	4.2 / SAD 3.4 Users; 4.2 SD-1; 4.3	Users and Reading	TC-U08	N	M3 Mukesh; Sprint 2
FR-U5	A user can set a reading status for a book: want to read, …	4.2 / SAD 3.4 Users; 4.2 SD-1; 4.3	Users and Reading	TC-U09	N	M3 Mukesh; Sprint 2
FR-U6	A user can record reading progress (page number or …	4.2 / SAD 3.4 Users; 4.2 SD-1; 4.3	Users and Reading	TC-U10, TC-U11	N	M3 Mukesh; Sprint 2
FR-U7	A user can view their reading history.	4.2 / SAD 3.4 Users; 4.2 SD-1; 4.3	Users and Reading	TC-U12	N	M3 Mukesh; Sprint 1–2
FR-U8	A user can see simple reading statistics (books finished, …	4.2 / SAD 3.4 Users; 4.2 SD-1; 4.3	Users and Reading	TC-U13	N	M3 Mukesh; Sprint 2
FR-B1	A logged-in member can reserve a book that has at least …	4.3 / SAD 3.4 Booking; 4.2 SD-2; 4.3	Booking	TC-B01, TC-B06	N	M4 Rushieswar; Sprint 1
FR-B2	A member can borrow a reserved book, which sets a due date …	4.3 / SAD 3.4 Booking; 4.2 SD-2; 4.3	Booking	TC-B07	N	M4 Rushieswar; Sprint 1
FR-B3	A member can return a borrowed book, which makes the copy …	4.3 / SAD 3.4 Booking; 4.2 SD-2; 4.3	Booking	TC-B03, TC-B12	N	M4 Rushieswar; Sprint 1
FR-B4	A member can cancel a reservation they have not yet …	4.3 / SAD 3.4 Booking; 4.2 SD-2; 4.3	Booking	TC-B08	N	M4 Rushieswar; Sprint 1
FR-B5	The available-copy count shall update automatically on …	4.3 / SAD 3.4 Booking; 4.2 SD-2; 4.3	Booking	TC-B01, TC-B03, TC-B05, TC-B08	N	M4 Rushieswar; Sprint 1
FR-B6	flag a booking as overdue when the due date has passed.	4.3 / SAD 3.4 Booking; 4.2 SD-2; 4.3	Booking	TC-B09	N	M4 Rushieswar; Sprint 2
FR-B7	A member can view their active and past bookings.	4.3 / SAD 3.4 Booking; 4.2 SD-2; 4.3	Booking	TC-B10	N	M4 Rushieswar; Sprint 2
FR-B8	A member shall not hold more than 5 active bookings at once.	4.3 / SAD 3.4 Booking; 4.2 SD-2; 4.3	Booking	TC-B04	N	M4 Rushieswar; Sprint 1
FR-B9	A member shall not be able to book a copy that is …	4.3 / SAD 3.4 Booking; 4.2 SD-2; 4.3	Booking	TC-B02	N	M4 Rushieswar; Sprint 1
FR-B10	A reservation that is not borrowed within 2 days shall …	4.3 / SAD 3.4 Booking; 4.2 SD-2; 4.3	Booking	TC-B05	N	M4 Rushieswar; Sprint 2
FR-R1	generate a ranked list of top-N book recommendations for a …	4.4 / SAD 3.4 Recommendation; 4.2 SD-3	Recommendation	TC-R01, TC-R08	N	M1 Sinan; Sprint 1

Req ID	Requirement (short)	Section ref / Design spec	Module	Test case(s)	Status	Owner / sprint
FR-R2	use the user's reading history (books read, progress, …	4.4 / SAD 3.4 Recommendation; 4.2 SD-3	Recommendation	TC-R02	N	M1 Sinan; Sprint 2
FR-R3	For a new user with no history, the system shall fall back …	4.4 / SAD 3.4 Recommendation; 4.2 SD-3	Recommendation	TC-R03	N	M1 Sinan; Sprint 1
FR-R4	not recommend books the user has already finished or books …	4.4 / SAD 3.4 Recommendation; 4.2 SD-3	Recommendation	TC-R04	N	M1 Sinan; Sprint 1
FR-R5	show a short reason for each recommendation (for example: …	4.4 / SAD 3.4 Recommendation; 4.2 SD-3	Recommendation	TC-R05	N	M1 Sinan; Sprint 2
FR-R6	An administrator can trigger a refresh or retrain of the …	4.4 / SAD 3.4 Recommendation; 4.2 SD-3	Recommendation	TC-R06	N	M1 Sinan; Sprint 2
NFR-C1	Search shall return results in under 1 second for 10,000 …	5 / SAD 3.4 Catalog; 4.3	Catalog	TC-P01	N	M2 Rishyendra; Sprint 2
NFR-C2	ISBN and required fields shall be validated, and duplicate …	5 / SAD 3.4 Catalog; 4.3	Catalog	TC-C11, TC-C12	N	M2 Rishyendra; Sprint 1
NFR-C3	The catalog module shall have at least 70% unit-test …	5 / SAD 3.4 Catalog; 4.3	Catalog	TC-Q01	N	M2 Rishyendra; Sprint 2
NFR-U1	Passwords shall be stored hashed (bcrypt or argon2), never …	5 / SAD 3.4 Users; 4.2 SD-1; 4.3	Users and Reading	TC-S01	N	M3 Mukesh; Sprint 1
NFR-U2	All protected endpoints shall reject requests without a …	5 / SAD 3.4 Users; 4.2 SD-1; 4.3	Users and Reading	TC-S02, TC-S03	N	M3 Mukesh; Sprint 1
NFR-U3	Email and password input shall be validated on the server.	5 / SAD 3.4 Users; 4.2 SD-1; 4.3	Users and Reading	TC-U03	N	M3 Mukesh; Sprint 1
NFR-U4	The Users and Reading module shall have at least 70% …	5 / SAD 3.4 Users; 4.2 SD-1; 4.3	Users and Reading	TC-Q01	N	M3 Mukesh; Sprint 2
NFR-B1	Booking actions shall be atomic: two users booking the …	5 / SAD 3.4 Booking; 4.2 SD-2; 4.3	Booking	TC-B11	N	M4 Rushieswar; Sprint 1–2
NFR-B2	Every booking action shall complete in under 1 second.	5 / SAD 3.4 Booking; 4.2 SD-2; 4.3	Booking	TC-P02	N	M4 Rushieswar; Sprint 2
NFR-B3	The booking module shall have at least 70% unit-test …	5 / SAD 3.4 Booking; 4.2 SD-2; 4.3	Booking	TC-Q01	N	M4 Rushieswar; Sprint 2
NFR-R1	A recommendation request shall respond in under 2 seconds …	5 / SAD 3.4 Recommendation; 4.2 SD-3	Recommendation	TC-P03	N	M1 Sinan; Sprint 2
NFR-R2	The algorithm shall sit behind a common interface so it …	5 / SAD 3.4 Recommendation; 4.2 SD-3	Recommendation	TC-R07	N	M1 Sinan; Sprint 1
NFR-R3	The recommendation module shall have at least 70% …	5 / SAD 3.4 Recommendation; 4.2 SD-3	Recommendation	TC-Q01	N	M1 Sinan; Sprint 2
NFR-G1	Every push and pull request shall run build, lint and …	5 / SAD 3.6; 4.4	CI/CD	TC-Q02	N	M3 Mukesh; Sprint 1–2
NFR-G2	The web UI shall be usable with a keyboard and a screen …	5 / SAD 3.6; 4.4	All modules	TC-UX01	N	All members; Sprint 2
SR-01	store passwords only as salted bcrypt hashes with a cost …	5.1.2 / SAD 3.4 Users; 4.2 SD-1; 4.3	Users and Reading	TC-S01	N	M3 Mukesh; Sprint 1
SR-02	Every protected endpoint shall require a signed JWT that …	5.1.2 / SAD 3.4 Users; 4.2 SD-1; 4.3	Users and Reading	TC-S02, TC-S03	N	M3 Mukesh; Sprint 1
SR-03	Administrator-only endpoints (book changes, recommendation …	5.1.2 / SAD 3.6; 4.4	All modules	TC-S05, TC-C09	N	All members; Sprint 1
SR-04	A member shall only be able to read or change their own …	5.1.2 / SAD 3.6; 4.4	All modules	TC-S06	N	All members; Sprint 1–2
SR-05	All input shall be validated on the server, and database …	5.1.2 / SAD 3.6; 4.4	All modules	TC-S04, TC-C12	N	All members; Sprint 1
SR-06	Secrets (JWT secret, admin password, database URL, deploy …	5.1.2 / SAD 3.6; 4.4	CI/CD	TC-S07	N	M3 Mukesh; Sprint 1
SR-07	The deployed application shall be served over HTTPS only.	5.1.2 / SAD 3.6; 4.4	CI/CD	TC-S08	N	M3 Mukesh; Sprint 2